

Universidad Tecnológica Nacional

Base de Datos II

Consultas de Acción en SQL Server

Autor: Angel Simón

Índice

1. Introducción	2
2. Diagrama Entidad-Relación	2
3. Script de Creación de la Base de Datos	3
4. Datos Iniciales	4
5. Consultas de Acción	5
5.1. INSERT	5
5.2. UPDATE	6
5.3. DELETE	7
6. Restricciones de Integridad y Errores Frecuentes	8
6.1. Violación de PRIMARY KEY	8
6.2. Violación de CHECK	8
6.3. Violación de FOREIGN KEY	9
7. Resumen	10

1. Introducción

En este apunte se trabaja con una base de datos orientada a registrar **ingresos y egresos personales**, denominada FinanzasPersonales. Este modelo sencillo resulta conveniente para ilustrar el funcionamiento de las **consultas de acción** en SQL Server, ya que involucra relaciones entre tablas, restricciones de integridad y operaciones de modificación de datos.

La base de datos almacena dos tipos de entidades: las **categorías** de movimientos (como sueldo, comida o alquiler) y los **movimientos** de dinero asociados a cada categoría.

Consultas de Acción vs. Consultas de Selección

Las consultas de acción se distinguen de las consultas SELECT en que **modifican los datos almacenados** en la base de datos. Los tres tipos principales son: INSERT, UPDATE y DELETE.

2. Diagrama Entidad-Relación

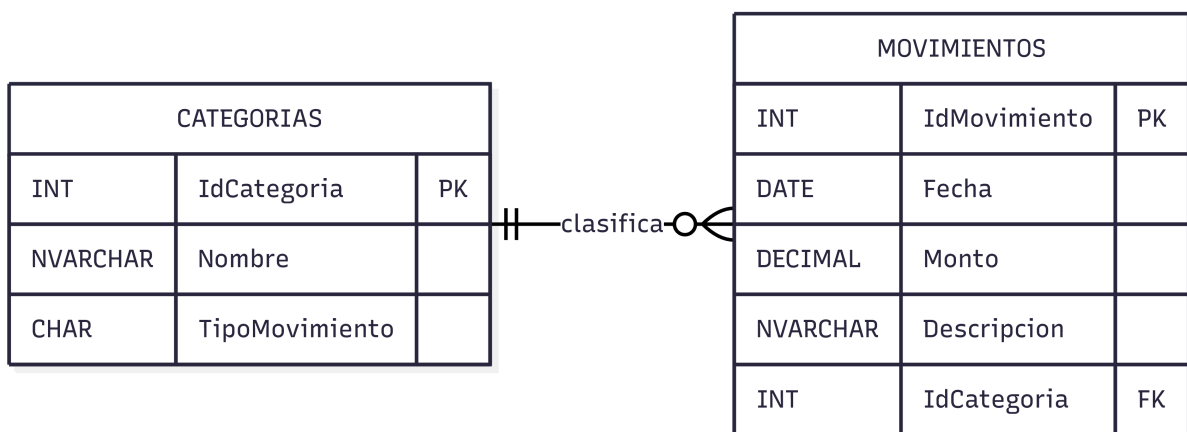


Figura 1. Diagrama de Entidad Relación de la Base de Datos FinanzasPersonales

El modelo de datos está compuesto por dos tablas relacionadas entre sí, tal como se describe en la **Tabla 1**.

Tabla	Atributo	Tipo / Restricción
Categorias	IdCategoria	INT PRIMARY KEY
	Nombre	NVARCHAR(100) NOT NULL
	TipoMovimiento	CHAR(1) CHECK IN ('I','E')
Movimientos	IdMovimiento	INT IDENTITY PK
	Fecha	DATE NOT NULL
	Monto	DECIMAL(12,2) NOT NULL
	Descripcion	NVARCHAR(200)
	IdCategoria	INT FOREIGN KEY

Tabla 1. Estructura de las tablas del modelo FinanzasPersonales

La relación entre ambas tablas es de **uno a muchos**: una categoría puede clasificar múltiples movimientos, mientras que cada movimiento pertenece a una única categoría.

3. Script de Creación de la Base de Datos

El siguiente script crea la base de datos, define las tablas con sus restricciones e inserta los datos iniciales de prueba.

```
CREATE DATABASE FinanzasPersonales;
GO

USE FinanzasPersonales;
GO

CREATE TABLE Categorias
(
    IdCategoria    INT PRIMARY KEY,
    Nombre         NVARCHAR(100) NOT NULL,
    TipoMovimiento CHAR(1) NOT NULL,

    CONSTRAINT CK_Categorias_TipoMovimiento
    CHECK (TipoMovimiento IN ('I','E'))
);
GO

CREATE TABLE Movimientos
(
    IdMovimiento  INT IDENTITY(1,1) PRIMARY KEY,
    Fecha         DATE NOT NULL,
    Monto         DECIMAL(12,2) NOT NULL,
    Descripcion   NVARCHAR(200),
    IdCategoria   INT NOT NULL,
```

```

CONSTRAINT FK_Movimientos_Categorias
FOREIGN KEY (IdCategoria)
REFERENCES Categorias(IdCategoria)
);
GO

INSERT INTO Categorias VALUES (1, 'Sueldo', 'I');
INSERT INTO Categorias VALUES (2, 'Freelance', 'I');
INSERT INTO Categorias VALUES (3, 'Alquiler', 'E');
INSERT INTO Categorias VALUES (4, 'Comida', 'E');
INSERT INTO Categorias VALUES (5, 'Transporte', 'E');

INSERT INTO Movimientos (Fecha, Monto, Descripcion, IdCategoria) VALUES
('2026-01-01', 3000, 'Cobro de sueldo', 1),
('2026-01-05', 1500, 'Supermercado', 4),
('2026-01-07', 80, 'Taxi', 5),
('2026-01-10', 1200, 'Pago alquiler', 3),
('2026-01-15', 1000, 'Trabajo freelance', 2);

```

Código 1. Creación de la base de datos e inserción de datos iniciales

4. Datos Iniciales

Antes de ejecutar cualquier consulta de acción, es importante conocer el estado inicial de las tablas. Las **Tablas 2** y **3** muestran los registros de partida.

IdCategoria	Nombre	TipoMovimiento
1	Sueldo	I
2	Freelance	I
3	Alquiler	E
4	Comida	E
5	Transporte	E

Tabla 2. Contenido inicial de la tabla *Categorias* (I = Ingreso, E = Egreso)

IdMovimiento	Fecha	Descripción	Monto	IdCategoria
1	2026-01-01	Cobro de sueldo	3 000	1
2	2026-01-05	Supermercado	100	4
3	2026-01-07	Taxi	80	5
4	2026-01-10	Pago alquiler	1 200	3
5	2026-01-15	Trabajo freelance	1 000	2

Tabla 3. Contenido inicial de la tabla Movimientos

5. Consultas de Acción

5.1. INSERT

La instrucción INSERT permite agregar nuevos registros a una tabla. La sintaxis básica especifica la tabla destino, las columnas a la cual se establecerán datos y los valores correspondientes.

Insertar un único registro:

```
INSERT INTO Categorías (IdCategoria, Nombre, TipoMovimiento)
VALUES (6, 'Servicios', 'E');
```

Código 2. Inserción de una categoría

Insertar múltiples registros en una sola sentencia:

```
INSERT INTO Categorías (IdCategoria, Nombre, TipoMovimiento)
VALUES
    (7, 'Internet', 'E'),
    (8, 'Regalos', 'E'),
    (9, 'Intereses', 'I');
```

Código 3. Inserción de múltiples categorías

Insertar registros a partir de una consulta SELECT:

Es posible combinar INSERT con SELECT para poblar una tabla con datos provenientes de otra consulta. En el siguiente ejemplo, se inserta un movimiento por cada categoría de tipo ingreso.

```
INSERT INTO Movimientos (Fecha, Monto, Descripción, IdCategoria)
SELECT
```

```

    GETDATE(),
    10000,
    'Ingreso adicional',
    IdCategoria
FROM Categorías
WHERE TipoMovimiento = 'I';

```

Código 4. Inserción mediante SELECT

5.2. UPDATE

La instrucción UPDATE permite modificar los valores de uno o más campos en registros existentes.

```

UPDATE Categorías
SET Nombre = 'Supermercado'
WHERE IdCategoria = 4;

```

Código 5. Actualización de una categoría

Modificación de varias columnas de una misma fila

Una consulta de UPDATE puede realizar más de una modificación sobre la misma fila. Por ejemplo:

```

UPDATE Movimientos
SET Descripcion='Supermercado', Fecha = '2026-02-19', Monto = 130
WHERE IdMovimiento=2;

```

Código 6. Actualización de la descripción, fecha y monto de un movimiento

Modificación de varias filas

Que una consulta de UPDATE realice más de una modificación sobre varias filas depende de la cláusula WHERE. En este ejemplo, se añaden \$10 a todos los movimientos del día 19-02-2026.

```

UPDATE Movimientos
SET Monto = Monto + 10
WHERE Fecha = '2026-02-19';

```

Código 7. Adición de 10 a los movimientos de una fecha determinada

Aquí, no hay ninguna garantía de la cantidad de registros que pueden ser modificados. Dependerá de cuántos registros tengamos para la fecha 19 de Febrero de 2026. Esto puede efectuar cambios en 0, 1 o varias filas.

Uso de WHERE en UPDATE

Omitir la cláusula WHERE provoca que la modificación se aplique a **todos los registros** de la tabla. La siguiente sentencia, por ejemplo, fijaría el monto en cero en todos los movimientos almacenados:

```
UPDATE Movimientos  
SET Monto = 0;
```

Esta operación es irreversible si no se cuenta con un respaldo previo.

5.3. DELETE

La instrucción DELETE elimina registros de una tabla de forma permanente.

```
DELETE FROM Movimientos  
WHERE IdMovimiento = 3;
```

Código 8. Eliminación de un movimiento específico

Uso de WHERE en DELETE

Al igual que con UPDATE, la ausencia de la cláusula WHERE resulta en la eliminación de **todos los registros** de la tabla:

```
DELETE FROM Movimientos;
```

Esta acción no elimina la estructura de la tabla, pero sí vacía completamente su contenido.

6. Restricciones de Integridad y Errores Frecuentes

SQL Server aplica restricciones de integridad para garantizar la consistencia de los datos. A continuación se describen los tres errores más habituales al trabajar con consultas de acción.

6.1. Violación de PRIMARY KEY

```
INSERT INTO Categorías VALUES (1, 'Servicios', 'E');
```

Código 9. Violación de Clave Primaria

La restricción de clave primaria asegura que cada registro sea identificable de forma unívoca. Intentar insertar un valor de `IdCategoría` ya existente provoca el siguiente error:

Error — PRIMARY KEY

Violation of PRIMARY KEY constraint 'PK__Categorías'.
The duplicate key value is (1).

Duplicación de clave primaria

Si al insertar registros manualmente se especifica un `IdCategoría` ya existente, la sentencia falla. Esto es especialmente frecuente cuando se trabaja con datos de prueba o se realiza una migración parcial de registros.

6.2. Violación de CHECK

```
INSERT INTO Categorías VALUES (10, 'Prueba', 'X');
```

Código 10. Violación de Check

La restricción CHECK limita los valores admitidos en una columna. En la tabla `Categorías`, el campo `TipoMovimiento` solo acepta los valores 'I' (ingreso) o 'E' (egreso). Intentar insertar un valor distinto, como 'X', produce el siguiente error:

Error — CHECK constraint

The INSERT statement conflicted with the CHECK constraint 'CK_Categorias_TipoMovimiento'.

6.3. Violación de FOREIGN KEY

```
INSERT INTO Movimientos (Fecha,Monto,Descripcion,IdCategoria)
VALUES ('2026-02-01',10000,'Compra',99);
```

Código 11. Violación de Foreign Key al insertar

La restricción de clave foránea garantiza que toda referencia a otra tabla apunte a un registro existente. Si se intenta insertar un movimiento con un IdCategoria que no existe en la tabla Categorias, SQL Server rechaza la operación:

Error — FOREIGN KEY (INSERT)

The INSERT statement conflicted with the FOREIGN KEY constraint 'FK_Movimientos_Categorias'.

El mismo tipo de error se produce al intentar eliminar una categoría que aún tiene movimientos asociados, ya que se rompería la integridad referencial:

```
DELETE FROM Categorias WHERE IdCategoria=4;
```

Código 12. Violación de Foreign Key al eliminar

Error — FOREIGN KEY (DELETE)

The DELETE statement conflicted with the REFERENCE constraint 'FK_Movimientos_Categorias'.

Orden de eliminación

Para eliminar una categoría que posee movimientos asociados, es necesario eliminar primero esos movimientos y luego la categoría. De lo contrario, SQL Server impedirá la operación para preservar la integridad referencial.

7. Resumen

Las consultas de acción constituyen las herramientas fundamentales para la manipulación de datos en SQL Server. La **Tabla 4** sintetiza su funcionamiento y las restricciones de integridad que pueden verse afectadas en cada caso.

Instrucción	Función
INSERT	Agrega nuevos registros
UPDATE	Modifica registros existentes
DELETE	Elimina registros

Tabla 4. Resumen de las consultas de acción y sus restricciones

Buenas prácticas

Antes de ejecutar cualquier sentencia UPDATE o DELETE, es recomendable realizar primero un SELECT con la misma condición WHERE para verificar qué registros se verán afectados. Esto previene modificaciones o eliminaciones accidentales en datos de producción.